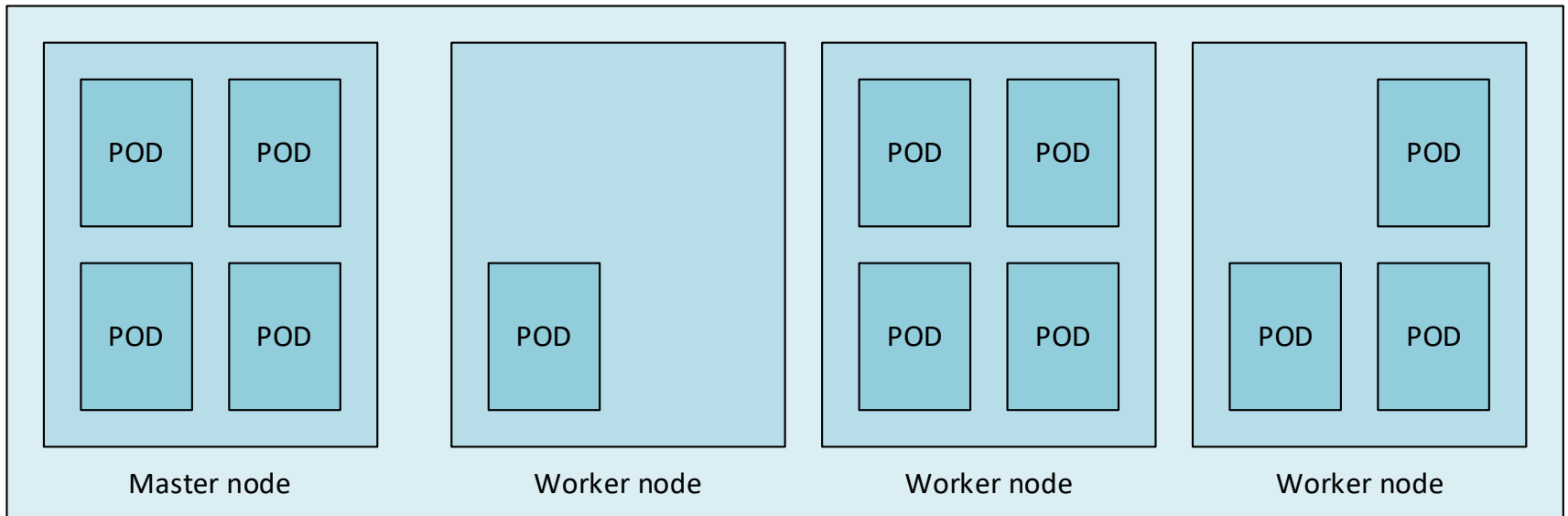


Kubernetes

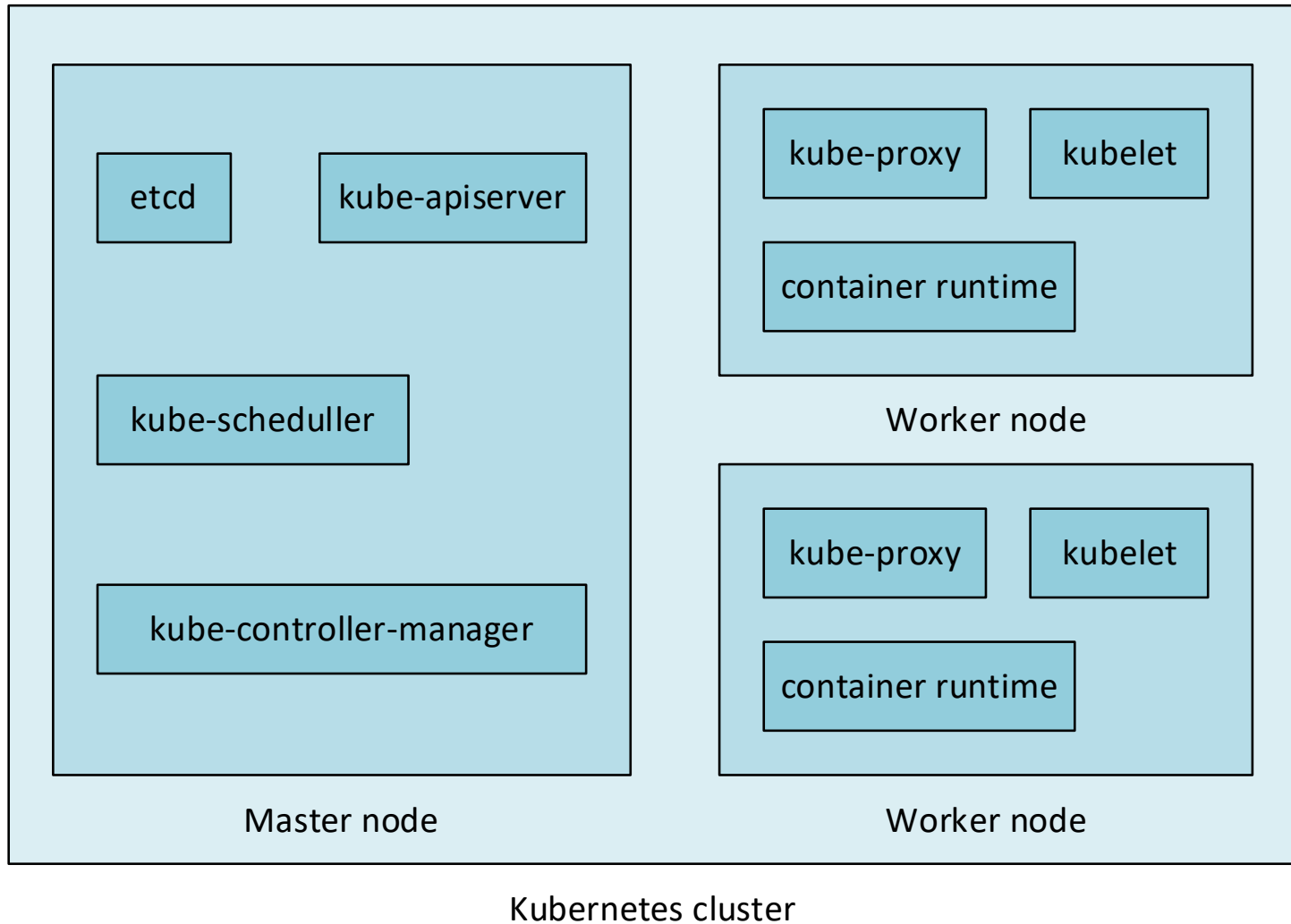
SAAR – Advanced Network Security and Architectures

Kubernetes

- An orchestrator for virtual containers (e.g., Docker), designed by Google (2014), now maintained by the Cloud Native Computing Foundation
- Key technology for the modern cloud (public, private, and hybrid)
- Pod: the smallest deployable unit of computing in Kubernetes; includes one or more containers, typically just one



Kubernetes components



Kubernetes components

- Control plane components:
 - **kube-apiserver** – Front-end for Kubernetes control plane, exposes the Kubernetes API
 - **etcd** – Stores all cluster data
 - **kube-scheduler** – Assigns containers to nodes, e.g., accounting for available resources and policies
 - **kube-controller-manager** – Runs the various controller processes (e.g., node controller is responsible for noticing and reacting when nodes go down, replication controller ensures that the desired number of containers are always running)

Kubernetes components

- Node components
 - **kubelet** – Manages the node and its containers, listening for instructions from the kube-apiserver
 - **kube-proxy** – Ensures that the necessary rules are in place at each worker node for communication between the containers running at these nodes
 - **container runtime** - Software responsible for running containers (e.g., Docker)
- Other components
 - **kubectl** – the CLI of Kubernetes
 - **DNS**

Kubernetes components

```
root@ub1:~# kubectl get pods -n kube-system -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
coredns-64897985d-s75t6	1/1	Running	11 (32m ago)	62d	10.32.0.2	ub1	<none>	<none>
coredns-64897985d-sl9dm	1/1	Running	11 (32m ago)	62d	10.32.0.3	ub1	<none>	<none>
etcd-ub1	1/1	Running	12 (32m ago)	62d	20.0.0.1	ub1	<none>	<none>
kube-apiserver-ub1	1/1	Running	12 (32m ago)	62d	20.0.0.1	ub1	<none>	<none>
kube-controller-manager-ub1	1/1	Running	28 (32m ago)	62d	20.0.0.1	ub1	<none>	<none>
kube-proxy-4q7gx	1/1	Running	12 (32m ago)	62d	20.0.0.3	ub3	<none>	<none>
kube-proxy-7457l	1/1	Running	12 (32m ago)	62d	20.0.0.2	ub2	<none>	<none>
kube-proxy-sp4nk	1/1	Running	11 (32m ago)	62d	20.0.0.1	ub1	<none>	<none>
kube-scheduler-ub1	1/1	Running	36 (32m ago)	62d	20.0.0.1	ub1	<none>	<none>
weave-net-dfm84	2/2	Running	30 (32m ago)	62d	20.0.0.2	ub2	<none>	<none>
weave-net-fdk8q	2/2	Running	27 (32m ago)	62d	20.0.0.3	ub3	<none>	<none>
weave-net-t5hd6	2/2	Running	27 (30m ago)	62d	20.0.0.1	ub1	<none>	<none>

Note: kubelet does not run as a POD

Ports and protocols

Control plane

Protocol	Direction	Port Range	Purpose	Used By
TCP	Inbound	6443	Kubernetes API server	All
TCP	Inbound	2379-2380	etcd server client API	kube-apiserver, etcd
TCP	Inbound	10250	Kubelet API	Self, Control plane
TCP	Inbound	10259	kube-scheduler	Self
TCP	Inbound	10257	kube-controller-manager	Self

Although etcd ports are included in control plane section, you can also host your own etcd cluster externally or on custom ports.

Worker node(s)

Protocol	Direction	Port Range	Purpose	Used By
TCP	Inbound	10250	Kubelet API	Self, Control plane
TCP	Inbound	30000-32767	NodePort Services†	All

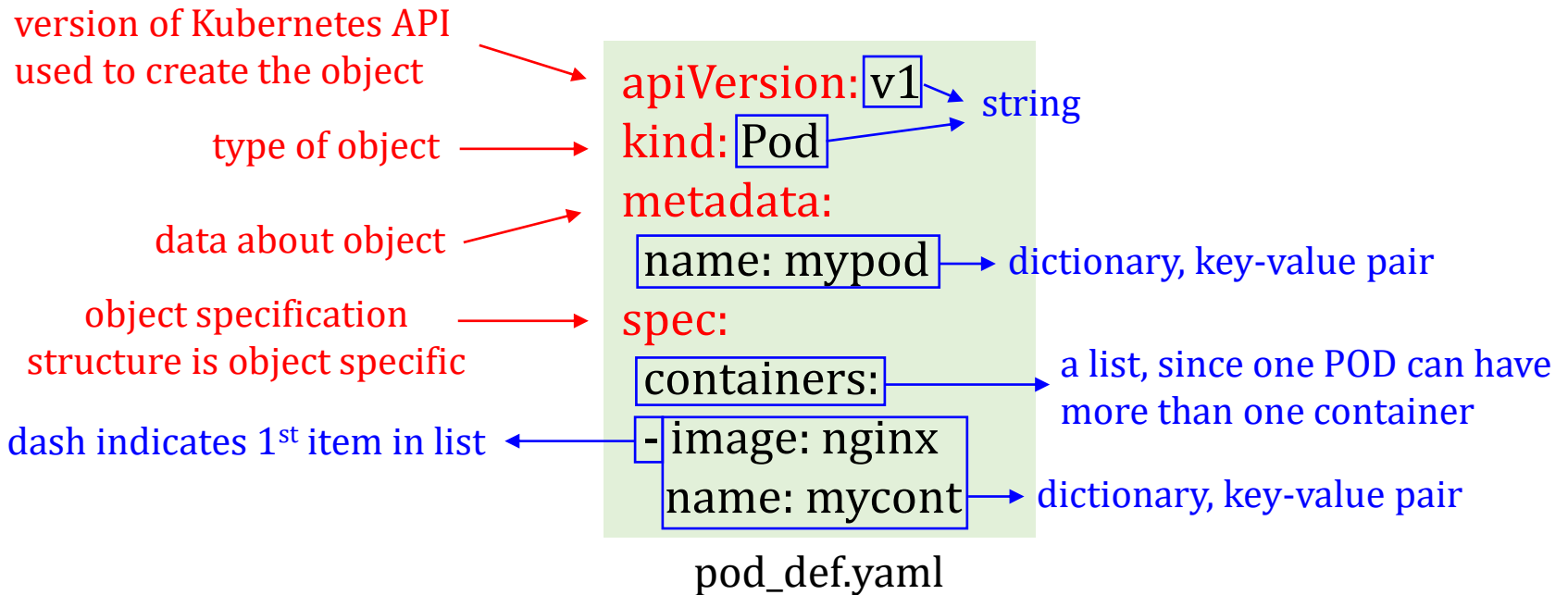
† Default port range for [NodePort Services](#).

Creating a POD

Using a single kubectl command:

```
kubectl run mypod --image=nginx
```

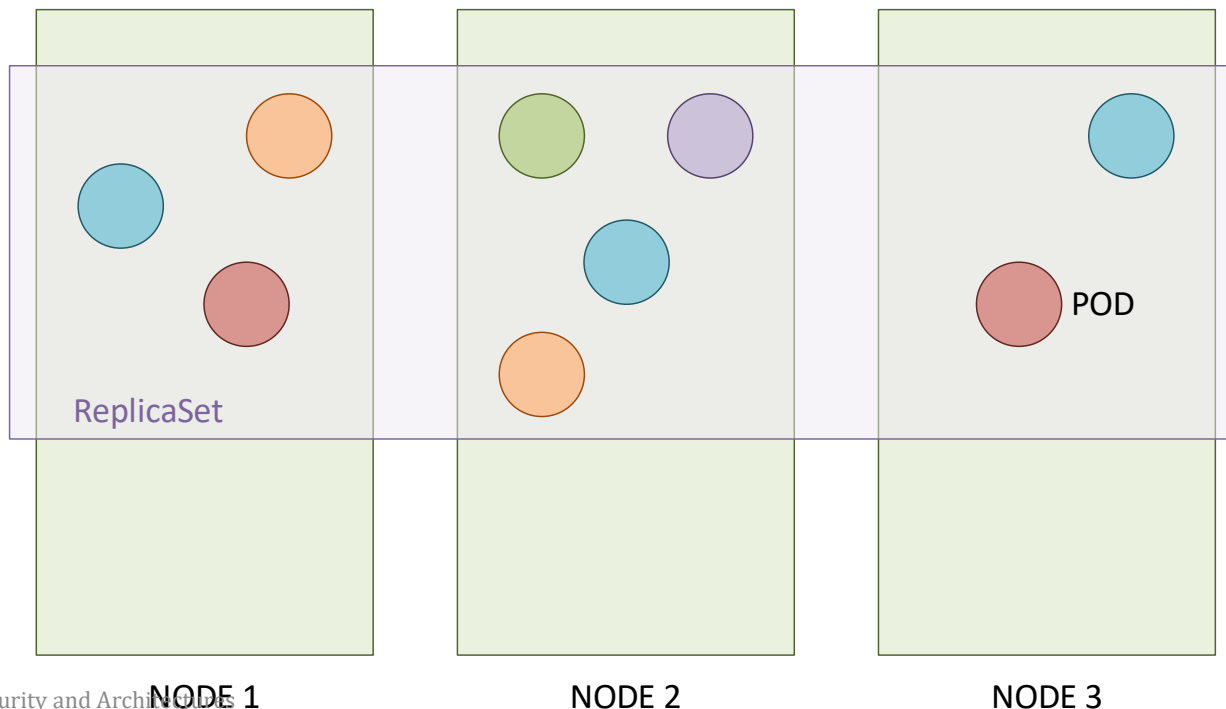
Using a YAML file and kubectl create (mandatory parts in red):



```
kubectl create -f pod_def.yaml
```

ReplicaSet

- The **ReplicaSet** is a **group** of containers spread over different nodes, that can be scaled up and down and has the self-healing property
- Self-healing: if one POD fails another is created, such that the number of PODS is kept constant, according to specified



Deploying a ReplicaSet

apiVersion: apps/v1

kind: ReplicaSet

metadata:

name: myrep

spec:

replicas: 5

selector:

matchLabels:

app: mynginx

template:

metadata:

labels:

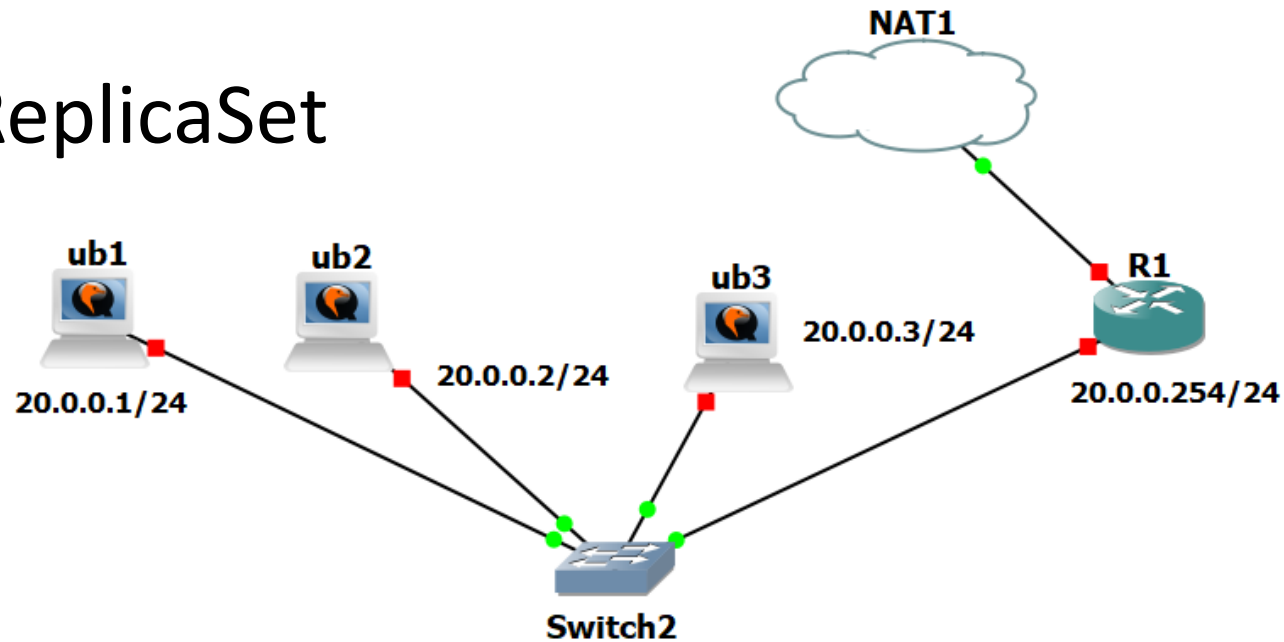
app: mynginx

spec:

containers:

- name: mycont

image: nginx



```
root@ub1:~# kubectl get pods -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP           NODE   NOMINATED NODE   READINESS GATES
myrep-bdscf   1/1     Running   0           87m   10.36.0.2    ub3    <none>            <none>
myrep-gjmvq   1/1     Running   0           87m   10.44.0.1    ub2    <none>            <none>
myrep-kh7bn   1/1     Running   0           87m   10.44.0.2    ub2    <none>            <none>
myrep-tndrw   1/1     Running   0           87m   10.36.0.3    ub3    <none>            <none>
myrep-vsclb   1/1     Running   0           87m   10.36.0.1    ub3    <none>            <none>
```

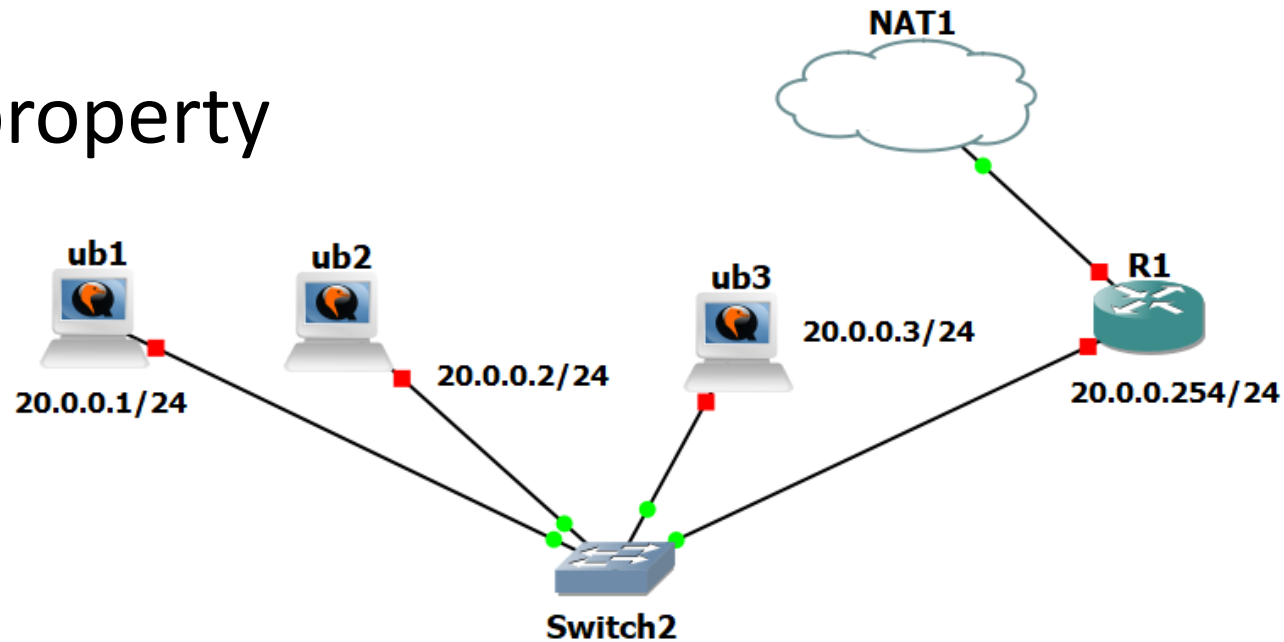
```
root@ub1:~# kubectl get replicaset -o wide
NAME          DESIRED   CURRENT   READY   AGE   CONTAINERS   IMAGES   SELECTOR
myrep         5         5         5       87m   mycont       nginx    app=mynginx
```

```
root@ub1:~# kubectl exec -it myrep-bdscf -- /bin/bash
root@myrep-bdscf:/# cat /etc/hosts
# Kubernetes-managed hosts file.
127.0.0.1    localhost
::1         localhost ip6-localhost ip6-loopback
fe00::0     ip6-localnet
fe00::0     ip6-mcastprefix
fe00::1     ip6-allnodes
fe00::2     ip6-allrouters
10.36.0.2    myrep-bdscf
root@myrep-bdscf:/# exit
exit
root@ub1:~#
```

Self-healing property

```

apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: myrep
spec:
  replicas: 5
  selector:
    matchLabels:
      app: mynginx
  template:
    metadata:
      labels:
        app: mynginx
    spec:
      containers:
        - name: mycont
          image: nginx
  
```



```

root@ub1:~# kubectl get pods -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE   NOMINATED NODE   READINESS GATES
myrep-bdscf   1/1     Running   0           132m  10.36.0.2     ub3    <none>            <none>
myrep-gjmvq   1/1     Running   0           132m  10.44.0.1     ub2    <none>            <none>
myrep-kh7bn   1/1     Running   0           132m  10.44.0.2     ub2    <none>            <none>
myrep-tndrw   1/1     Running   0           132m  10.36.0.3     ub3    <none>            <none>
myrep-vsclb   1/1     Running   0           132m  10.36.0.1     ub3    <none>            <none>

root@ub1:~# kubectl delete pods myrep-vsclb
pod "myrep-vsclb" deleted

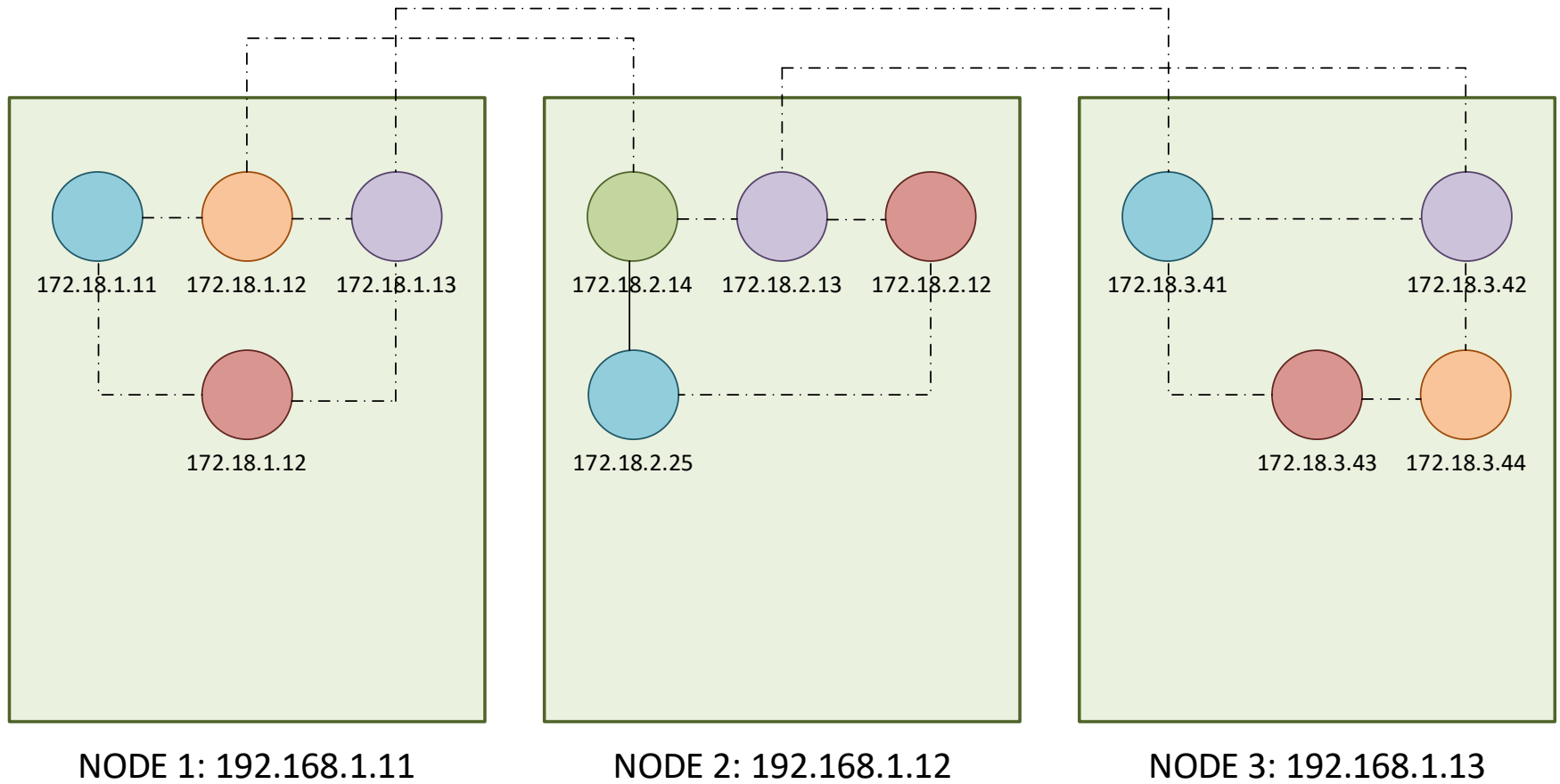
root@ub1:~# kubectl get pods -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE   NOMINATED NODE   READINESS GATES
myrep-bdscf   1/1     Running   0           134m  10.36.0.2     ub3    <none>            <none>
myrep-g2fpv   1/1     Running   0           69s   10.44.0.3     ub2    <none>            <none>
myrep-gjmvq   1/1     Running   0           134m  10.44.0.1     ub2    <none>            <none>
myrep-kh7bn   1/1     Running   0           134m  10.44.0.2     ub2    <none>            <none>
myrep-tndrw   1/1     Running   0           134m  10.36.0.3     ub3    <none>            <none>
  
```

A pink arrow points from the deleted pod 'myrep-vsclb' in the first table to the newly created pod 'myrep-g2fpv' in the second table, demonstrating the self-healing property where a new pod is created to replace the deleted one.

POD networking

- POD networking requirements:
 1. Every POD should have an IP address
 2. Every POD should be able to communicate with every other POD in the same node
 3. Every POD should be able to communicates with every other POD on other nodes without NAT
- Kubernetes **does not** provide a built-in POD networking solution
- It relies on the **CNI (Container Network Interface)** standard
- Many CNI solutions available: **Weave**, Flannel, Calico, Cilium,...

POD networking

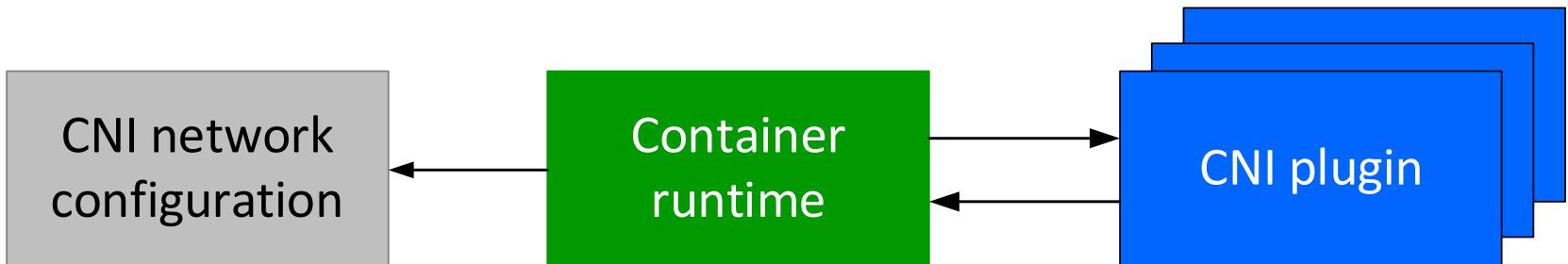


Container Network Interface (CNI)

- [CNI](#) is a Cloud Native Computing Foundation project
- Goal of the CNI project:
 - To provide a **standard method** for attaching a container to a network and for applying the network configuration that is required for this purpose, both to the host and inside the container
- Not used exclusively by Kubernetes

Container Network Interface (CNI)

- Invoked when a container is created
- The container runtime looks for the **CNI network configuration file** (JSON format), which tells which **CNI plugins** must be invoked, and then invokes them one by one.

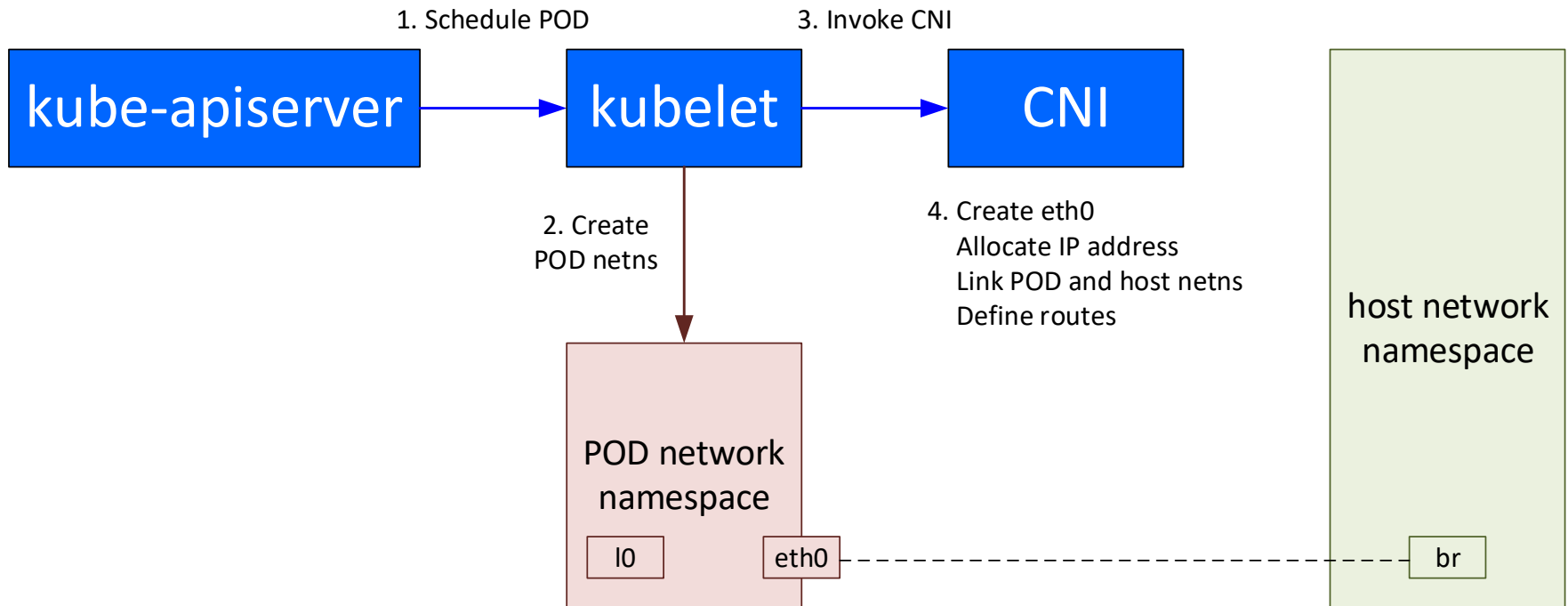


Container Network Interface (CNI)

- In Kubernetes, the CNI network configuration file can be found in the directory **/etc/cni/net.d** and the available CNI plugins in the directory **/opt/cni/bin**.

```
root@ub1:~# ls /opt/cni/bin
bandwidth  firewall      loopback  sbr        vrf
bridge     host-device  macvlan   static     weave-ipam
dhcp       host-local   portmap   tuning     weave-net
dummy      ipvlan       ptp       vlan       weave-plugin-latest
root@ub1:~# ls /etc/cni/net.d
10-weave.conflist
root@ub1:~# cat /etc/cni/net.d/10-weave.conflist
{
  "cniVersion": "0.3.0",
  "name": "weave",
  "plugins": [
    {
      "name": "weave",
      "type": "weave-net",
      "hairpinMode": true
    },
    {
      "type": "portmap",
      "capabilities": {"portMappings": true},
      "snat": true
    }
  ]
}
```

CNI in Kubernetes



CNI in Kubernetes

- The CNI plugin:
 - Manages local connectivity
- A daemon:
 - Manages reachability across the cluster

```
103 - apiVersion: extensions/v1beta1
104   kind: DaemonSet
105   metadata:
106     name: weave-net
107     labels:
108       name: weave-net
109     namespace: kube-system
110   spec:
111     # Wait 5 seconds to let pod connect before rolling next pod
112     minReadySeconds: 5
113     template:
114       metadata:
115         labels:
116           name: weave-net
117       spec:
118         initContainers:
119           - name: weave-init
120             image: 'weaveworks/weave-kube:latest'
121             imagePullPolicy: Always
122             command:
123               - /home/weave/init.sh
124             env:
125               securityContext:
126                 privileged: true
127             volumeMounts:
128               - name: cni-bin
129                 mountPath: /host/opt
130               - name: cni-bin2
131                 mountPath: /host/home
132               - name: cni-conf
133                 mountPath: /host/etc
```

weave DaemonSet definition (extract)

CNI in Kubernetes

- One Weave agent (POD) in every node to handle networking

```
root@ub1:~# kubectl get pods -n kube-system -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE	READINESS	GATES
coredns-64897985d-s75t6	1/1	Running	11 (32m ago)	62d	10.32.0.2	ub1	<none>		<none>	
coredns-64897985d-sl9dm	1/1	Running	11 (32m ago)	62d	10.32.0.3	ub1	<none>		<none>	
etcd-ub1	1/1	Running	12 (32m ago)	62d	20.0.0.1	ub1	<none>		<none>	
kube-apiserver-ub1	1/1	Running	12 (32m ago)	62d	20.0.0.1	ub1	<none>		<none>	
kube-controller-manager-ub1	1/1	Running	28 (32m ago)	62d	20.0.0.1	ub1	<none>		<none>	
kube-proxy-4q7gx	1/1	Running	12 (32m ago)	62d	20.0.0.3	ub3	<none>		<none>	
kube-proxy-7457l	1/1	Running	12 (32m ago)	62d	20.0.0.2	ub2	<none>		<none>	
kube-proxy-sp4nk	1/1	Running	11 (32m ago)	62d	20.0.0.1	ub1	<none>		<none>	
kube-scheduler-ub1	1/1	Running	36 (32m ago)	62d	20.0.0.1	ub1	<none>		<none>	
weave-net-dfm84	2/2	Running	30 (32m ago)	62d	20.0.0.2	ub2	<none>		<none>	
weave-net-fdk8q	2/2	Running	27 (32m ago)	62d	20.0.0.3	ub3	<none>		<none>	
weave-net-t5hd6	2/2	Running	27 (30m ago)	62d	20.0.0.1	ub1	<none>		<none>	

IP Address Management (IPAM)

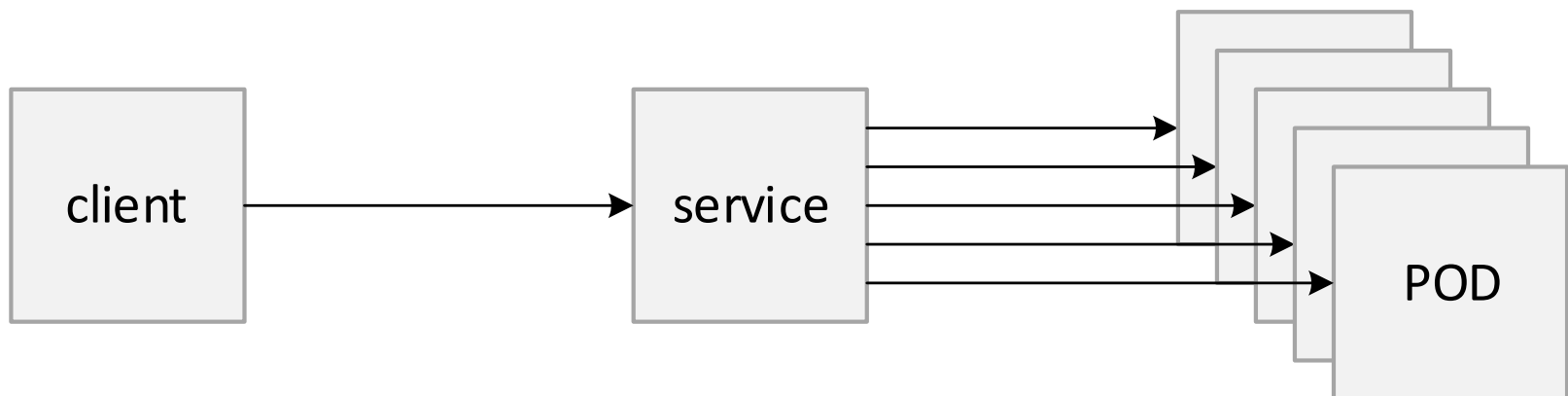
- The CNI plugin is responsible for assigning IP addresses to PODs in a unique way
- Weave uses the range 10.32.0.0/12 (10.32.0.1-10.47.255.254) and assigns non-overlapping subranges to each node

```
root@ub1:~# kubectl get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
myrep1-7nzdt	1/1	Running	3 (151m ago)	21h	10.36.0.3	ub2
myrep1-d5wwr	1/1	Running	3 (151m ago)	21h	10.32.0.2	ub3
myrep1-j56vl	1/1	Running	0	7m59s	10.36.0.5	ub2
myrep1-kgmbd	1/1	Running	3 (151m ago)	21h	10.36.0.2	ub2
myrep1-ncnjc	1/1	Running	0	8m	10.32.0.5	ub3
myrep1-phvds	1/1	Running	0	8m	10.32.0.4	ub3
myrep1-plhxb	1/1	Running	0	8m	10.36.0.4	ub2
myrep1-qm7t4	1/1	Running	3 (151m ago)	21h	10.32.0.3	ub3
myrep1-qsxfw	1/1	Running	0	7m59s	10.32.0.6	ub3
myrep1-sc5d9	1/1	Running	3 (151m ago)	21h	10.36.0.1	ub2
myrep2-fnv92	1/1	Running	0	2m13s	10.36.0.7	ub2
myrep2-q6nl9	1/1	Running	0	2m13s	10.32.0.8	ub3
myrep2-r2hb4	1/1	Running	0	2m13s	10.32.0.7	ub3
myrep2-svhkv	1/1	Running	0	2m13s	10.36.0.6	ub2
myrep2-z9gzj	1/1	Running	0	2m13s	10.32.0.9	ub3

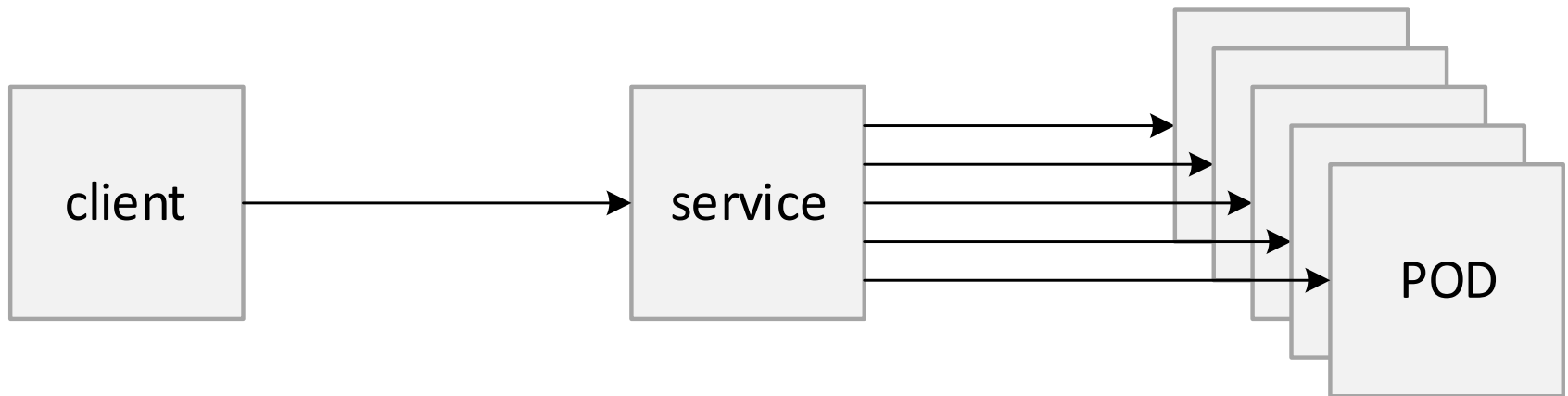
Kubernetes services

- Provides a way of abstracting access to a group of PODs (the service endpoints)
- From a networking perspective, a service consists of two parameters: IP address and port number.
 - Service IP addresses cannot overlap with POD IP addresses
- When a client connects to a service, the connection is load balanced to one of the pods backing the service
- The group of pods backing each service is usually defined using a label selector



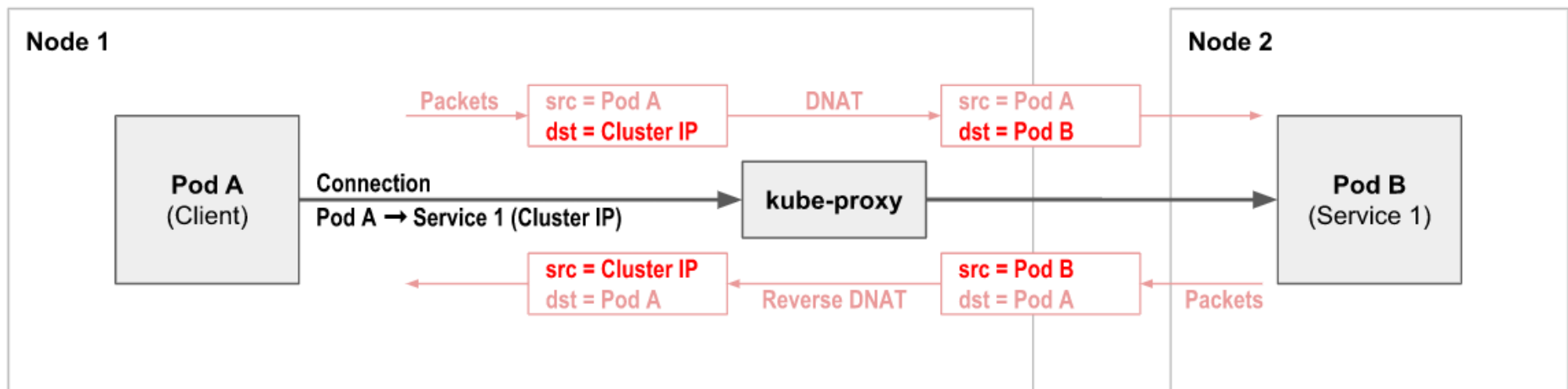
Kubernetes services

- Main service types (handled by kube-proxy)
 - **ClusterIP** - the usual way of accessing a service from inside the cluster
 - **NodePort** - the most basic way of accessing a service from outside the cluster
 - **LoadBalancer** - uses an external load balancer as a more sophisticated way to access a service from outside the cluster



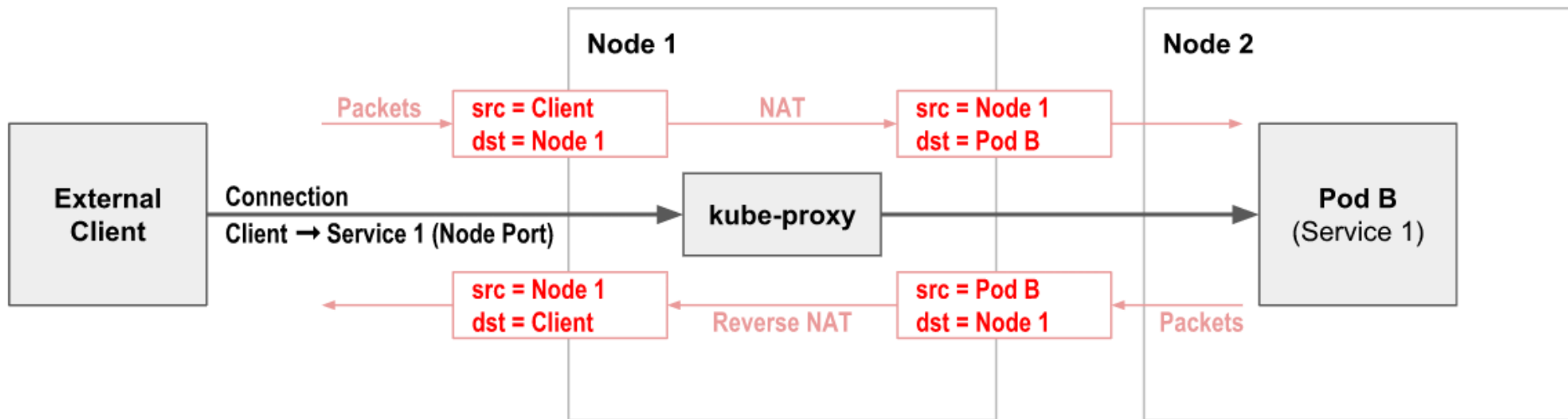
ClusterIP service

- Default service type, suitable for **intra-cluster** communication
- Provides virtual IP address, the Cluster IP address, for accessing the service, which is discoverable through the Kubernetes DNS
- The DNS name and Cluster IP address remain constant for the lifetime of the service, even if the PODs backing the service come and go

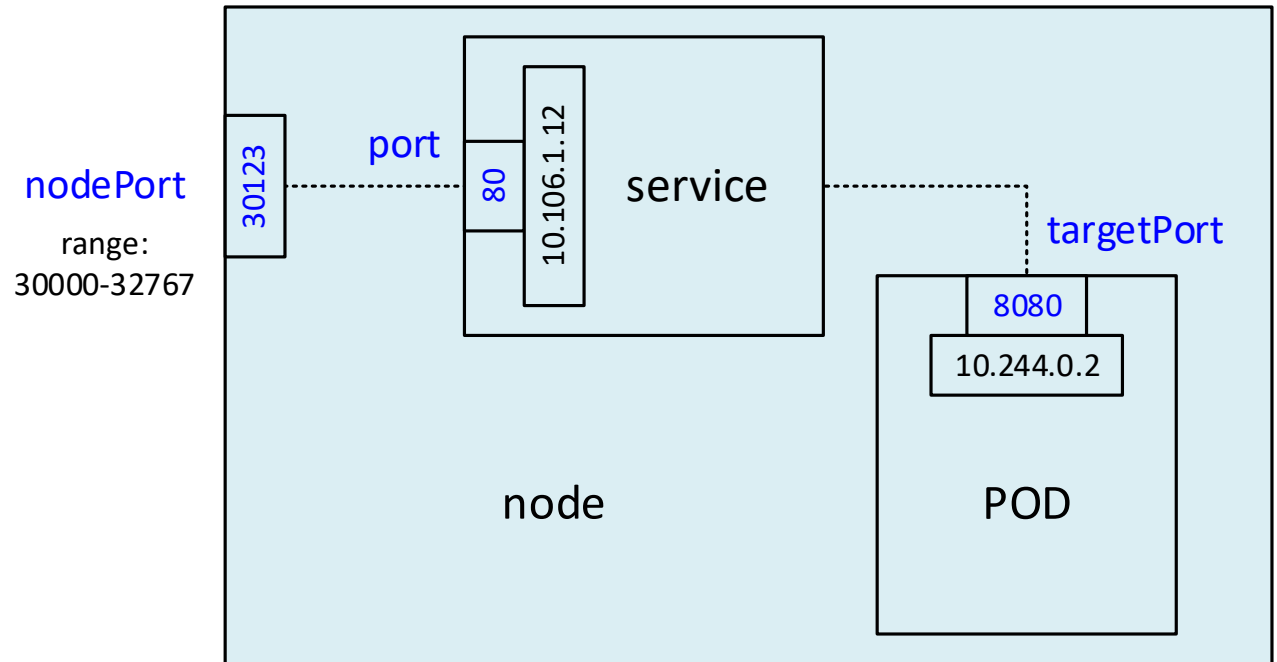


NodePort service

- A port reserved on each node in the cluster through which the service can be accessed

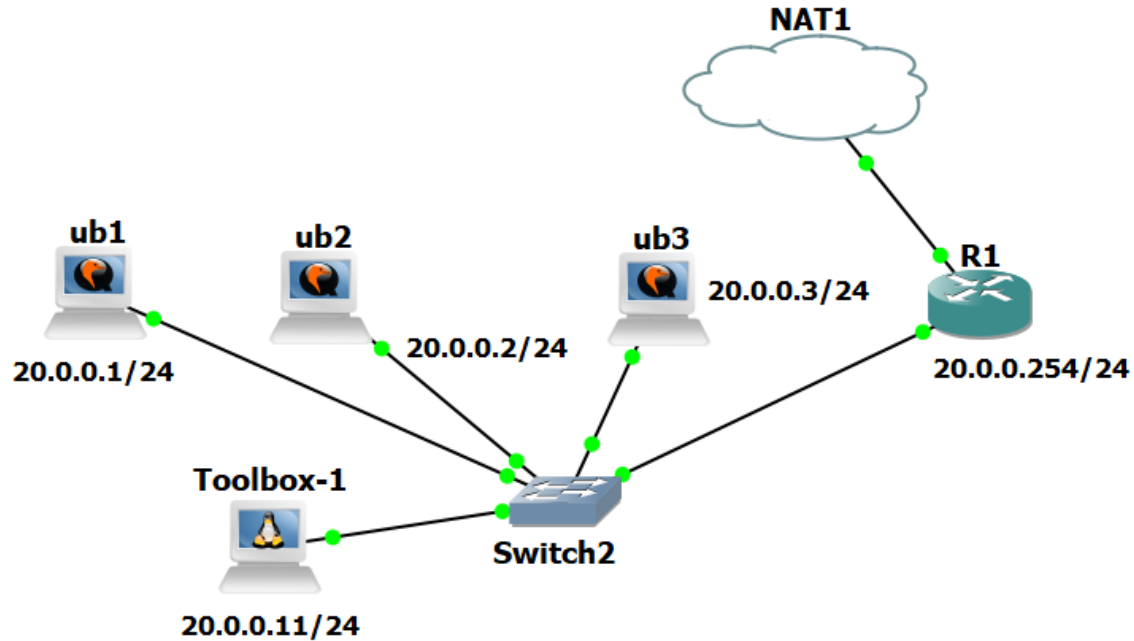


NodePort service



```
apiVersion: v1
kind: Service
metadata:
  name: mysvc
spec:
  type: NodePort
  ports:
    - port: 80
      targetPort: 8080
      nodePort: 30123
  selector:
    app: mykubia
```

NodePort



```
apiVersion: apps/v1
```

```
kind: ReplicaSet
```

```
metadata:
```

```
  name: myrep
```

```
spec:
```

```
  replicas: 3
```

```
  selector:
```

```
    matchLabels:
```

```
      app: mykubia
```

```
  template:
```

```
    metadata:
```

```
      labels:
```

```
        app: mykubia
```

```
  spec:
```

```
    containers:
```

```
      - name: mycont
```

```
        image: kubia
```

```
        imagePullPolicy: Never
```

```
apiVersion: v1
```

```
kind: Service
```

```
metadata:
```

```
  name: mysvc
```

```
spec:
```

```
  type: NodePort
```

```
  ports:
```

```
    - port: 80
```

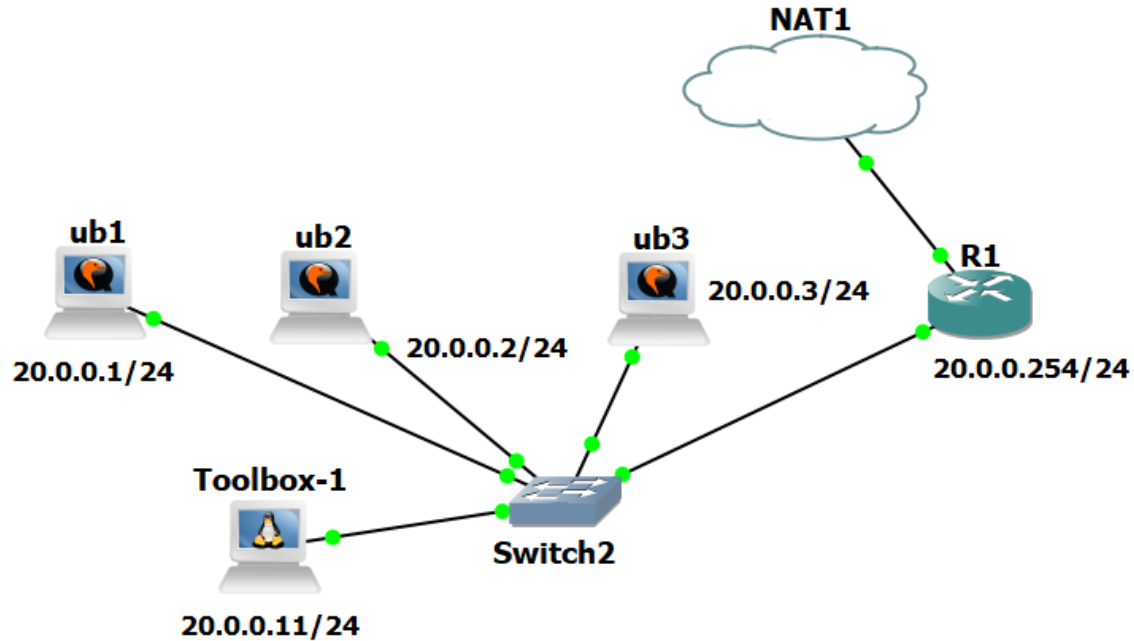
```
      targetPort: 8080
```

```
      nodePort: 30123
```

```
  selector:
```

```
    app: mykubia
```

NodePort

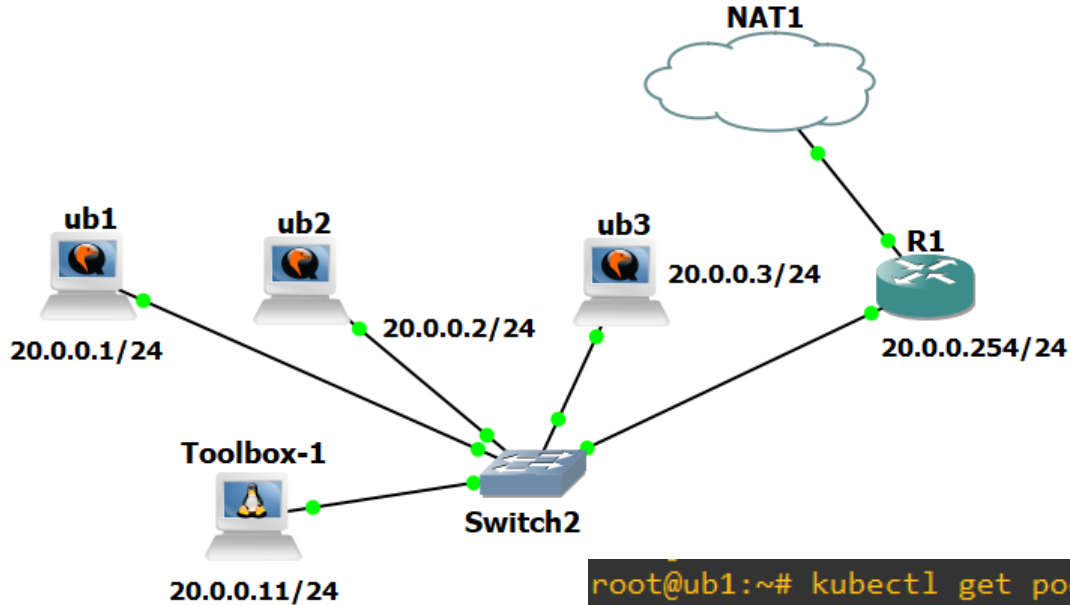


```
root@ub1:~# kubectl get endpoints mysvc
NAME      ENDPOINTS                                     AGE
mysvc     10.36.0.1:8080,10.36.0.2:8080,10.44.0.1:8080 11h
```

```
root@ub1:~# kubectl get svc -o wide
NAME          TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE    SELECTOR
kubernetes    ClusterIP     10.96.0.1     <none>         443/TCP          65d    <none>
mysvc         NodePort      10.108.17.108 <none>         80:30123/TCP     7m7s   app=mykubia
```

```
root@ub1:~# kubectl get pods -o wide
NAME          READY   STATUS    RESTARTS   AGE    IP          NODE    NOMINATED NODE    READINESS GATES
myrep-6d8n5   1/1     Running   0          10m    10.36.0.1   ub3     <none>             <none>
myrep-rq8mj   1/1     Running   0          10m    10.36.0.2   ub3     <none>             <none>
myrep-s7lx7   1/1     Running   0          10m    10.44.0.1   ub2     <none>             <none>
```

NodePort



```
root@ub1:~# kubectl get pods -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP           NODE
myrep-6d8n5   1/1     Running   1 (9m19s ago)  11h   10.36.0.1    ub3
myrep-rq8mj   1/1     Running   1 (9m19s ago)  11h   10.36.0.2    ub3
myrep-s7lx7   1/1     Running   1 (9m26s ago)  11h   10.44.0.1    ub2
root@ub1:~# kubectl exec -it myrep-6d8n5 -- /bin/bash
root@myrep-6d8n5:/# ping -c 2 10.36.0.2
PING 10.36.0.2 (10.36.0.2): 56 data bytes
64 bytes from 10.36.0.2: icmp_seq=0 ttl=64 time=0.319 ms
64 bytes from 10.36.0.2: icmp_seq=1 ttl=64 time=0.597 ms
--- 10.36.0.2 ping statistics ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max/stddev = 0.319/0.458/0.597/0.139 ms
root@myrep-6d8n5:/# ping -c 2 10.44.0.1
PING 10.44.0.1 (10.44.0.1): 56 data bytes
64 bytes from 10.44.0.1: icmp_seq=0 ttl=64 time=4.973 ms
64 bytes from 10.44.0.1: icmp_seq=1 ttl=64 time=5.622 ms
--- 10.44.0.1 ping statistics ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max/stddev = 4.973/5.297/5.622/0.325 ms
root@myrep-6d8n5:/# exit
exit
```

NodePort – load balancing

5	20.867690	20.0.0.11	20.0.0.3	HTTP	144 GET / HTTP/1.1
7	20.951523	20.0.0.3	20.0.0.11	HTTP	208 HTTP/1.1 200 OK
15	25.287565	20.0.0.11	20.0.0.3	HTTP	144 GET / HTTP/1.1
17	25.292581	20.0.0.3	20.0.0.11	HTTP	208 HTTP/1.1 200 OK
27	27.561561	20.0.0.11	20.0.0.3	HTTP	144 GET / HTTP/1.1
29	27.612715	20.0.0.3	20.0.0.11	HTTP	208 HTTP/1.1 200 OK
37	29.911571	20.0.0.11	20.0.0.3	HTTP	144 GET / HTTP/1.1
39	29.928451	20.0.0.3	20.0.0.11	HTTP	208 HTTP/1.1 200 OK
47	32.022471	20.0.0.11	20.0.0.3	HTTP	144 GET / HTTP/1.1
49	32.039465	20.0.0.3	20.0.0.11	HTTP	208 HTTP/1.1 200 OK
57	33.944850	20.0.0.11	20.0.0.3	HTTP	144 GET / HTTP/1.1
59	33.949474	20.0.0.3	20.0.0.11	HTTP	208 HTTP/1.1 200 OK
67	35.486191	20.0.0.11	20.0.0.3	HTTP	144 GET / HTTP/1.1
69	35.496202	20.0.0.3	20.0.0.11	HTTP	208 HTTP/1.1 200 OK
77	37.583998	20.0.0.11	20.0.0.3	HTTP	144 GET / HTTP/1.1
79	37.593682	20.0.0.3	20.0.0.11	HTTP	208 HTTP/1.1 200 OK
87	47.463003	20.0.0.11	20.0.0.2	HTTP	144 GET / HTTP/1.1
89	47.488142	20.0.0.2	20.0.0.11	HTTP	208 HTTP/1.1 200 OK
97	51.678972	20.0.0.11	20.0.0.2	HTTP	144 GET / HTTP/1.1
99	51.685832	20.0.0.2	20.0.0.11	HTTP	208 HTTP/1.1 200 OK
107	52.837323	20.0.0.11	20.0.0.2	HTTP	144 GET / HTTP/1.1
109	52.848161	20.0.0.2	20.0.0.11	HTTP	208 HTTP/1.1 200 OK
117	54.961827	20.0.0.11	20.0.0.2	HTTP	144 GET / HTTP/1.1
119	54.968829	20.0.0.2	20.0.0.11	HTTP	208 HTTP/1.1 200 OK
127	56.415506	20.0.0.11	20.0.0.2	HTTP	144 GET / HTTP/1.1
129	56.424801	20.0.0.2	20.0.0.11	HTTP	208 HTTP/1.1 200 OK
137	57.536402	20.0.0.11	20.0.0.2	HTTP	144 GET / HTTP/1.1
139	57.548984	20.0.0.2	20.0.0.11	HTTP	208 HTTP/1.1 200 OK
147	58.519408	20.0.0.11	20.0.0.2	HTTP	144 GET / HTTP/1.1
149	58.557454	20.0.0.2	20.0.0.11	HTTP	208 HTTP/1.1 200 OK
158	60.812899	20.0.0.11	20.0.0.2	HTTP	144 GET / HTTP/1.1
160	60.817755	20.0.0.2	20.0.0.11	HTTP	208 HTTP/1.1 200 OK
168	62.653304	20.0.0.11	20.0.0.2	HTTP	144 GET / HTTP/1.1
170	62.707694	20.0.0.2	20.0.0.11	HTTP	208 HTTP/1.1 200 OK

```

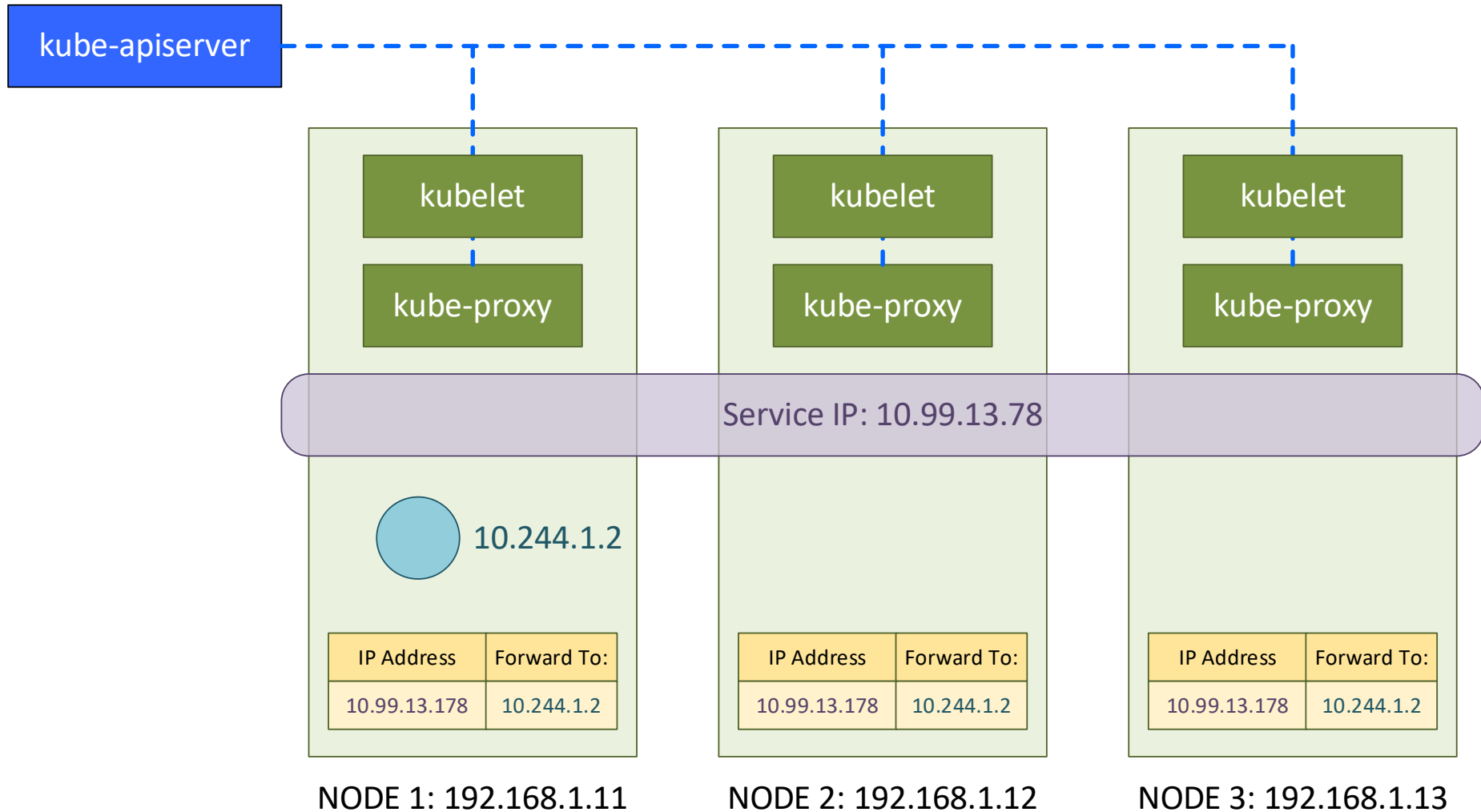
root@Toolbox-1:~# curl http://20.0.0.3:30123
You've hit myrep-s7lx7
root@Toolbox-1:~# curl http://20.0.0.3:30123
You've hit myrep-s7lx7
root@Toolbox-1:~# curl http://20.0.0.3:30123
You've hit myrep-6d8n5
root@Toolbox-1:~# curl http://20.0.0.3:30123
You've hit myrep-s7lx7
root@Toolbox-1:~# curl http://20.0.0.3:30123
You've hit myrep-rq8mj
root@Toolbox-1:~# curl http://20.0.0.3:30123
You've hit myrep-s7lx7
root@Toolbox-1:~# curl http://20.0.0.3:30123
You've hit myrep-rq8mj
root@Toolbox-1:~# curl http://20.0.0.3:30123
You've hit myrep-s7lx7
root@Toolbox-1:~# curl http://20.0.0.2:30123
You've hit myrep-6d8n5
root@Toolbox-1:~# curl http://20.0.0.2:30123
You've hit myrep-s7lx7
root@Toolbox-1:~# curl http://20.0.0.2:30123
You've hit myrep-6d8n5
root@Toolbox-1:~# curl http://20.0.0.2:30123
You've hit myrep-6d8n5
root@Toolbox-1:~# curl http://20.0.0.2:30123
You've hit myrep-6d8n5
root@Toolbox-1:~# curl http://20.0.0.2:30123
You've hit myrep-6d8n5
root@Toolbox-1:~# curl http://20.0.0.2:30123
You've hit myrep-s7lx7
root@Toolbox-1:~# curl http://20.0.0.2:30123
You've hit myrep-rq8mj
root@Toolbox-1:~# curl http://20.0.0.2:30123
You've hit myrep-s7lx7

```

Kubernetes services details

- The default service implementation is using **kube-proxy**
 - Alternative: eBPF
- Three kube-proxy **modes**:
 - **userspace**
 - Legacy
 - **iptables**
 - Designed for firewalling
 - A list of rules organized in tables and chains
 - Good performance in most cases
 - **ipvs**
 - Designed for load balancing
 - Based on the Linux IP Virtual Server, an advanced L4 load balancer
 - Relies on in-kernel hash tables
 - Appropriate for large clusters

Kubernetes services details



Kubernetes services details

```
-A PREROUTING -m comment --comment "kubernetes service portals" -j KUBE-SERVICES
-A KUBE-SERVICES -d 10.96.0.10/32 -p udp -m comment --comment "kube-system/kube-dns:dns cluster IP" -m udp --dport 53 -j KUBE-SVC-TCOU7JCQXEZGVUNU
-A KUBE-SERVICES -d 10.96.0.10/32 -p tcp -m comment --comment "kube-system/kube-dns:dns-tcp cluster IP" -m tcp --dport 53 -j KUBE-SVC-ERIFXISQEP7F70F4
-A KUBE-SERVICES -d 10.96.0.10/32 -p tcp -m comment --comment "kube-system/kube-dns:metrics cluster IP" -m tcp --dport 9153 -j KUBE-SVC-JD5MR3NA4I4DYORP
-A KUBE-SERVICES -d 10.106.50.158/32 -p tcp -m comment --comment "default/mysvc1 cluster IP" -m tcp --dport 80 -j KUBE-SVC-K3WEUCSSBHSPA422
-A KUBE-SERVICES -d 10.96.0.1/32 -p tcp -m comment --comment "default/kubernetes:https cluster IP" -m tcp --dport 443 -j KUBE-SVC-NPX46M4PTMTKRN6Y
-A KUBE-SERVICES -m comment --comment "kubernetes service nodeports; NOTE: this must be the last rule in this chain" -m addrtype --dst-type LOCAL -j KUBE-SVC-K3WEUCSSBHSPA422
-A KUBE-SVC-K3WEUCSSBHSPA422 ! -s 10.32.0.0/12 -d 10.106.50.158/32 -p tcp -m comment --comment "default/mysvc1 cluster IP" -m tcp --dport 80 -j KUBE-MARK-MASQ
-A KUBE-SVC-K3WEUCSSBHSPA422 -m comment --comment "default/mysvc1 -> 10.32.0.2:80" -m statistic --mode random --probability 0.500000000000 -j KUBE-SEP-5SGYSBICXIG4SBSQ
-A KUBE-SVC-K3WEUCSSBHSPA422 -m comment --comment "default/mysvc1 -> 10.36.0.1:80" -j KUBE-SEP-HG2XYTPQNHHMK2EVP
-A KUBE-SEP-HG2XYTPQNHHMK2EVP -s 10.36.0.1/32 -m comment --comment "default/mysvc1" -j KUBE-MARK-MASQ
-A KUBE-SEP-HG2XYTPQNHHMK2EVP -p tcp -m comment --comment "default/mysvc1" -m tcp -j DNAT --to-destination 10.36.0.1:80
-A KUBE-SEP-5SGYSBICXIG4SBSQ -s 10.32.0.2/32 -m comment --comment "default/mysvc1" -j KUBE-MARK-MASQ
-A KUBE-SEP-5SGYSBICXIG4SBSQ -p tcp -m comment --comment "default/mysvc1" -m tcp -j DNAT --to-destination 10.32.0.2:80
```

```
root@ub1:~# kubectl get pods -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP          NODE   NOMINATED NODE   READINESS GATES
myrep1-qwc2k  1/1     Running   0           22m   10.36.0.1   ub2    <none>            <none>
myrep1-zx8h8  1/1     Running   0           22m   10.32.0.2   ub3    <none>            <none>
```

```
root@ub1:~# kubectl get svc
NAME          TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE
kubernetes    ClusterIP   10.96.0.1    <none>        443/TCP    3d20h
mysvc1        ClusterIP   10.106.50.158 <none>        80/TCP     3h41m
```

iptables rules for ClusterIP service (10.106.50.158:80) with two PODs (10.36.0.1 and 10.32.0.2)